An Industrial Training/Internship (AIM 4170) **Execution Methodology** on

Claims Auto-Adjudication Engine for Health Insurance

Carried out at Niva Bupa Health Insurance Company Limited, Gurugram

Submitted to Manipal University Jaipur

Towards the partial fulfilment for the Award of the Degree of

B. Tech Computer Science and Engineering (Artificial Intelligence and Machine Learning)

In Computer Science and Engineering (AIML)

2026-2027

By

Manvik Talwar – 23FE10CAI00130



MANIPAL UNIVERSITY
JAIPUR

Under the guidance of

Dr. Noopur Tyagi

Department of Artificial Intelligence and Machine Learning

Manipal University Jaipur

Jaipur, Rajasthan

CONTENTS

1. Introduction 1
2. Methodology at a Glance 1
3. Requirement Analysis and Planning 2
   3.1 Sources of Requirements 2
   3.2 From Clause to Rule 2
   3.3 Scope Definition 3
   3.4 Planning 3
4. Phase-wise Execution 3
5. Week-by-Week Log 4
6. Development Workflow 5
   6.1 The Per-Rule Loop 5
   6.2 Working Practice 6
7. Tools and Environment 6
8. Verification and Validation 6
9. Risk Management 7
10. Communication and Review 7
11. Outcomes against Objectives 8
   11.1 Reflection on the Methodology 8

1. Introduction

This document describes **how** the internship project was executed: how requirements were gathered and analysed, how the work was planned and divided into phases, what was done each week, how quality was assured and risks were handled, and how progress was reviewed. The design and implementation of the system itself are described in the accompanying training report.

The project was a claims auto-adjudication engine for Niva Bupa's ReAssure 3.0 health insurance product. Given a claim line item and the policy, member and benefit data, the engine decides whether to approve, partially approve, reject or refer the claim to a human reviewer, computes the payable amount, and records the reasoning behind every step. Deterministic rules handle eligibility and money; a large language model (LLM) is used only to interpret clinical text.

Table 1: *Internship details*

Item	Details
Student	Manvik Talwar (23FE10CAI00130), B.Tech CSE (AIML), 7th semester
Course	Industrial Training/Internship (AIM 4170), 2026-2027
Organisation	Niva Bupa Health Insurance Company Limited, Gurugram head office
Team	IT Applications
Duration	20 May 2026 to 20 July 2026 (9 weeks)
Reporting managers	Sudhanshu Jain, Rajat Gupta
Faculty supervisor	Dr. Noopur Tyagi, Department of AIML, MUJ
Nature of work	Solo project: analysis, design, implementation, testing and documentation

2. Methodology at a Glance

The work followed an **iterative, incremental and specification-driven** model. The specification (the policy wording and the technical solution design) was fixed at the start; the system was then built in phases, each adding one capability on top of a working, tested base. Each phase had a defined exit check before the next began.

Table 2: *Key methodological choices*

Aspect	Choice	Reason
Life-cycle model	Iterative and incremental, phase-gated	Each phase ends in a working system; problems surface early rather than at integration
Requirements	Specification-driven: policy wording and technical solution design	Insurance decisions must follow the contract exactly; the documents are the source of truth

Aspect	Choice	Reason
Order of building	Deterministic engine first, AI second	Exact logic was proven before any AI was added, so the AI could never hide a logic error
Rules	Encoded as versioned data (rule blueprints), not hard-coded	Product changes become data changes; rules are reviewable by non-programmers
Use of AI	LLM only for clinical language, with validated output and human fallback	LLMs can be wrong with confidence; money and eligibility must be exact
Quality	Unit, property-based, scenario, API and end-to-end tests	Different defect types need different kinds of test
Debugging	Trace-driven: read every decision trace and reasoning log	Most defects showed up as wrong reasoning, not failed assertions
Version control	Git with separate stable and working branches	Safe experimentation and a traceable history
Documentation	Written alongside the code at each phase	Handover-ready at the end of the internship

3. Requirement Analysis and Planning

3.1 Sources of Requirements

Table 3: *Requirement sources studied before coding*

Source	What it contained	How it was used
ReAssure 3.0 policy wording	2,600+ lines: covers, exclusions, waiting periods, room-rent and co-payment clauses, annexures	Ground truth for every rule and formula
Technical solution design (27 May 2026)	Gate sequence, tool contracts, state model, confidence routing, error handling	Architecture and execution order
Product and policy rules extraction	92 rule blueprints, rate tables, mutual-exclusivity constraints	Loaded as the product memory
Product and policy API specification	Shapes of the 8 upstream data APIs	Context builder and simulated gateways

3.2 From Clause to Rule

Each clause of the policy wording was analysed and converted into a **rule blueprint**: an identifier, the gate it belongs to, whether it is deterministic, semantic (needs language understanding) or hybrid, the variants it applies to, the rules it depends on, the calculator it needs, and for semantic rules a prompt that states the rule and its exceptions. Clauses with exceptions received the most attention, because that is where both people and simple keyword matching go wrong (for example, cosmetic surgery is excluded unless it is reconstruction after an accident, burns or cancer).

3.3 Scope Definition

The scope was fixed early to the product, policy and member validation layer: the engine receives line items already extracted from bills and medical documents and returns a decision per line item. The Policy Administration System remains the final authority that pays. Fraud analytics, hospital tariffs and upstream extraction were kept out of scope; a document-upload feature was added only to demonstrate field extraction.

3.4 Planning

The work was split into phases with an exit check for each (Section 4). The plan was deliberately ordered so that every phase depended only on phases already verified: data models before calculators, calculators before the pipeline, the deterministic pipeline before the AI layer, the engine before the API and user interface.

4. Phase-wise Execution

Table 4 summarises each phase and Table 5 shows when each phase ran. Phases overlapped slightly, because the first activities of a phase usually began as the previous phase was being closed.

Table 4: *Phases, activities and exit checks*

Phase	Main activities	Output	Exit check
0. Domain study and planning	Onboarding in IT Applications; study of the claims process, policy wording and technical solution design; rule inventory; project plan	Rule inventory, phase plan, project structure	Open questions on the wording resolved with reporting managers
1. Deterministic core	Pydantic data models; eight calculators (waiting period, pro-rata, co-pay, deductible, SI waterfall, Lock the Clock, Booster+, pre/post window); seven-gate sequential pipeline; developer front end	Working deterministic engine	Numbers for sample claims checked by hand; calculator unit tests pass
2. AI layer	Product memory loading the rule blueprints; DAG planner with topological sort; semantic agent on a local LLM; four-tier confidence routing	Engine that plans per claim and reads clinical text	Component-by-component compliance check against the design document
3. Hardening and observability	Agent reasoning logger; JSONL telemetry; property-based tests; interactive decision-graph export	Full audit trail; invariant tests	Every LLM call and rule result traceable; invariants hold over random inputs
4. Integration and data layer	Context builder for 8 gateway APIs and simulated gateways; PostgreSQL schema, Alembic migrations, Docker Compose	Claim context assembled from APIs; persistent state	Claim assembled and adjudicated end to end

Phase	Main activities	Output	Exit check
5. Product user interface	Design system; React console; live Server-Sent Events stream; OCR document extraction; AI decision summaries	Demonstrable console	Ten scenario presets run from the UI
6. Refactor and documentation	Pipeline split into gate modules (execution, financials, state, validation); multi-line claims; endorsements; technical specification	Maintainable code base; specification document	Same results as before the refactor (regression suite)
7. Lifetime state and handover	ReAssure Forever state machine; financial validation; architecture and handover document	Handover package	Handover document reviewed
8. Final hardening (before submission)	Cloud LLM integration (Groq); batched LLM screening; end-to-end scenario testing on the live LLM; defect fixes; report	77 tests; 10 live scenarios	All tests pass; all scenarios match the deterministic results

Table 5: *Gantt chart of the phases*

Phase	W1	W2	W3	W4	W5	W6	W7	W8	W9	Oct
0. Domain study and planning										
1. Deterministic core										
2. AI layer										
3. Hardening and observability										
4. Integration and data layer										
5. Product user interface										
6. Refactor and documentation										
7. Lifetime state and handover										
8. Final hardening										

W1 = 20–26 May 2026 … W9 = 15–20 Jul 2026; Oct = final hardening before submission.

5. Week-by-Week Log

Table 6 records what was done in each week of the internship and what it produced. Dates of commits and documents in the project repository are given in parentheses.

Table 6: *Weekly log*

Week	Dates	Work done	Output
W1	20–26 May	Joined the IT Applications team at the Gurugram head office. Studied the health claims process and the ReAssure 3.0 product: variants, optional covers, Booster+, ReAssure Forever, Lock the Clock.	Domain notes; list of open questions
W2	27 May–2 Jun	Studied the technical solution design (dated 27 May) and the policy wording clause by clause; reviewed the rule extraction and mapped every rule to a gate; planned the phases; designed the data models and the project structure.	Rule-to-gate mapping; project structure document (2 Jun)

Week	Dates	Work done	Output
W3	3–9 Jun	Set up the repository (first commit 3 Jun) with schemas, calculators, the seven-gate pipeline, product memory, planner and a first semantic agent. Built a developer front end connected to the API (5 Jun); wrote the first architecture document; added the agent reasoning logger (8 Jun).	Working deterministic engine; developer UI; architecture draft
W4	10–16 Jun	Checked the implementation against every section of the design document (compliance report, 10 Jun); added property-based tests; added telemetry; built the context builder and simulated gateway APIs (12 Jun); reorganised the code base.	Compliance report; invariant tests; integration layer
W5	17–23 Jun	Added interactive decision-graph export and debugging aids (17 Jun); ran the scenario claims end to end and fixed issues found by reading the traces.	Decision graphs; scenario runs
W6	24–30 Jun	Integrated the API specification for the eight data sources; added PostgreSQL with Alembic migrations and Docker Compose (26 Jun); defined the design system and the console layout (30 Jun).	Database layer; design system
W7	1–7 Jul	Restructured the product memory layer (2 Jul); added OCR-based document extraction (2 Jul); built the live streaming console and its state management (6 Jul); updated the architecture document.	Streaming console; document upload
W8	8–14 Jul	Refactored the pipeline into separate gate modules (14 Jul); added multi-line claims and endorsement handling; wrote the technical specification document (14 Jul).	Modular pipeline; technical specification
W9	15–20 Jul	Implemented the ReAssure Forever state machine and financial validation (16 Jul); wrote the architecture and handover document (16 Jul); handed over the project.	Handover document
Final	Oct 2026	Before submission: integrated the Groq cloud LLM, batched exclusion screening to stay within free-tier limits, ran all ten scenarios on the live LLM, fixed the defects found, and wrote the report.	77 passing tests; report, presentation

6. Development Workflow

6.1 The Per-Rule Loop

Every rule went through the same five steps. Repeating one loop for 92 rules kept the work consistent and made progress easy to measure (rules implemented and tested out of the total).

1. **Analyse** the clause in the policy wording, including its exceptions and cross-references.
2. **Encode** it as a rule blueprint: gate, execution type, dependencies, applicable variants, calculator or prompt.
3. **Implement** the gate check or calculator, keeping calculators pure functions so they are easy to test.

4. **Test** it: a unit test; for money, a property-based test; and a scenario preset that exercises it.
5. **Read the trace**: run the scenario and read the decision trace and LLM reasoning log line by line; fix and repeat.

6.2 Working Practice

- Work was committed to Git in small steps, with a stable branch and a working branch.
- The full test suite was run before each commit; it runs offline in about 30 seconds because a deterministic responder stands in for the LLM.
- Questions about the meaning of a clause were collected and resolved with the reporting managers rather than guessed.
- Documentation (architecture, compliance report, specification, handover) was updated at the end of the phase it described.

7. Tools and Environment

Table 7: *Tools used during execution*

Purpose	Tools
Language and validation	Python 3.12, Pydantic v2, pydantic-settings
Service layer	FastAPI, Uvicorn, asyncio, Server-Sent Events
AI	Local LLM through llama.cpp (Gemma 4); Groq API (Qwen3.8 27B)
Documents	pypdf, Tesseract OCR, Pillow
Data	PostgreSQL 16, SQLAlchemy 2 (async), Alembic, Docker Compose, pgAdmin
User interface	React 19, TypeScript, Vite, Tailwind CSS
Testing	pytest, Hypothesis, FastAPI TestClient, Playwright
Engineering	Git and GitHub, VS Code, uv

8. Verification and Validation

Verification checked that each part was built correctly; validation checked that the whole system made the right decisions on realistic claims.

Table 8: *Verification and validation activities*

Level	What was checked	Result
Unit	Each calculator and gate against hand-worked examples	Pass
Property-based	Money conserved; ratios bounded; pools never negative; deductible within limit	Pass over thousands of random inputs

Level	What was checked	Result
Scenario	Complete claims through the pipeline against expected outcomes	Pass
API and provider	Endpoints, streaming, LLM provider, batching and fallback	Pass
Compliance	Each component against each section of the design document	Gaps recorded (e.g. PAS reconciliation is a stub)
End to end	Ten scenario presets through the real UI on the live LLM	10/10 complete; decisions match the deterministic mode

In total 77 automated tests pass. End-to-end testing in the final phase found defects that unit tests had missed, including a room-rent deduction that could exceed the claimed amount and a request for human review that was being lost; each fix came with a regression test that fails on the old code.

9. Risk Management

Table 9: *Risks identified and how they were handled*

Risk	Impact	Mitigation
Misreading a policy clause	Wrong decisions on real claims	Clause-by-clause analysis; exceptions written into rules; questions resolved with reporting managers
LLM gives a confident wrong answer	Wrong exclusion or approval	LLM only for language; schema-validated output; confidence thresholds; human review on any doubt
Arithmetic errors in money	Over- or under-payment	Deterministic calculators; property-based tests of invariants
Cloud LLM rate limits or outages	Slow or failed adjudication	Batched screening (one call per line item); failures route to review; offline mode for demonstrations
Sensitive health data	Privacy exposure	Simulated data during development; local-model option for production; minimal fields in prompts
Concurrency bugs	Inconsistent state between claims	Per-claim and thread-local state; row locks on balances
Scope creep	Unfinished core	Scope fixed to the validation layer; extras (document upload) kept optional

10. Communication and Review

Progress and open questions were reviewed with the reporting managers, Sudhanshu Jain and Rajat Gupta, who explained the claims domain, clarified the policy wording and reviewed design decisions. Written artefacts were produced at the end of the phases they describe, so the state of the project was always documented:

Table 10: *Documents produced during the internship*

Document	When	Purpose
Project structure	2 Jun	Planned modules and phase boundaries
Architecture document	5 Jun, updated to 6 Jul	System design reference
Compliance report	10 Jun	Implementation checked against the design document
Design system	30 Jun	Consistent user interface
Technical specification	14 Jul	Detailed specification of the engine
Architecture and handover document	16 Jul	Handover to the team
Training report, presentation	Oct 2026	Academic submission

11. Outcomes against Objectives

Table 11: *Objectives and how they were met*

Objective	Outcome
Encode the product rules as data	92 rule blueprints in a versioned product memory
Exact money	Deterministic calculators with property-based tests
LLM only for language	Semantic agent with validated JSON output; batched screening
Safe routing	Four-tier confidence routing; data gaps and errors go to human review
Complete audit trail	Decision trace, tool calls, LLM prompts and replies, telemetry, decision graphs
Usable product	Streaming API and React console; cloud, local and offline LLM modes
Thorough testing	77 automated tests; 10 scenarios on the live LLM

11.1 Reflection on the Methodology

Building the deterministic engine before adding AI was the most valuable decision: it gave a trusted baseline, and in the end the live-LLM results matched it on every scenario. Reading traces end to end found more defects than assertions did. If I repeated the project, I would build a labelled evaluation set from historical claims at the start, so that every change could be measured as agreement with human decisions, and I would add end-to-end scenario runs to the routine of every phase rather than mainly at the end.